BI-VWM.21 – R-strom

Pavel Holman, Matyáš Volf

Duben 2024

1 Popis projektu

Cílem projektu je implementovat R-strom tj. databázovou indexovací strukturu pro vícerozměrné objekty a porovnat ji se sekvenčním průchodem celé databáze. [2]

2 Způsob řešení

K řešení tohoto tématu byla implementována struktura R-strom, inspirovaná referencí ze zadání [3]. Zároveň projekt obsahuje jednoduché grafické rozhraní, které poskytuje prostředí pro postavení vlastního R-stromu a sledování výstupu jednotlivých operací.

Strom je z části konfigurovatelný. Je možné si například zvolit kapacitu jednotlivých uzlů, nebo ponechat výchozí nastavení, které je optimalizované pro velikost stránek disku. Dále je možné vybrat strategii, pomocí které dochází k rozdělení jednotlivých uzlů, dojde-li k jejich přeplnění. Nakonec je možné i měnit dimenzi indexovaných objektů stromu.

3 Implementace

Celý projekt je implementován v jazyce Rust [4]. Obsahuje 3 části:

- Implementaci R-stromu,
- Grafické rozhraní,
- Testy výkonu.

Samotná implementace poskytuje rozhraní, pomocí kterého je možné do stromu vkládat prvky, vyhledávat prvky dle indexu a vyhledávat K nejbližších sousedů. Dále lze strom ukládat na disk a poté ho i z disku načíst. Nakonec je poskytnuto i rozhraní, které umožní sledovat celou vnitřní strukturu stromu, ale bez možnosti změny vnitřního stavu – ta by totiž mohla vést k narušení invariantů stromu.

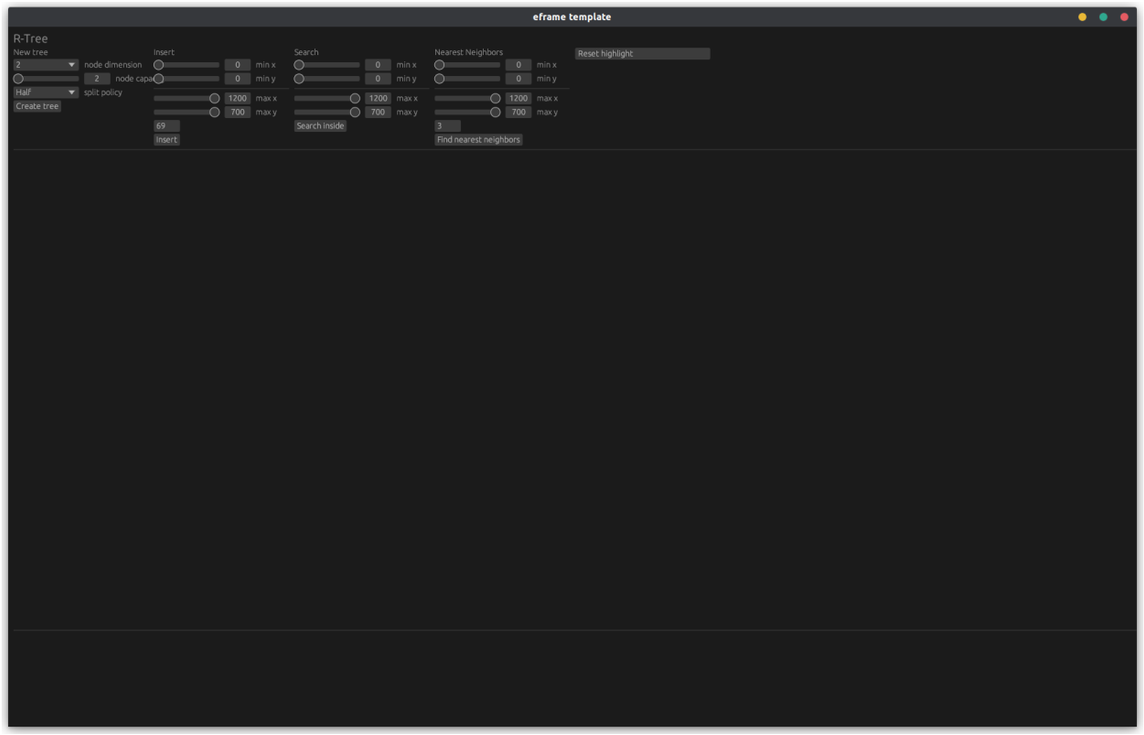
Grafické rozhraní je implementováno pomocí knihovny egui [1]. Poskytuje jak konfiguraci stromu, (dimenze objektů, strategie dělení, kapacita uzlů), tak i prostředí pro testování jednotlivých operací, tedy vkládání, vyhledávání dle indexu a vyhledávání K nejbližších sousedů. Navíc je možné strom uložit a opět načíst.

Testy výkonu porovnávají strom se sekvenčním průchodem pro experimentální sekci.

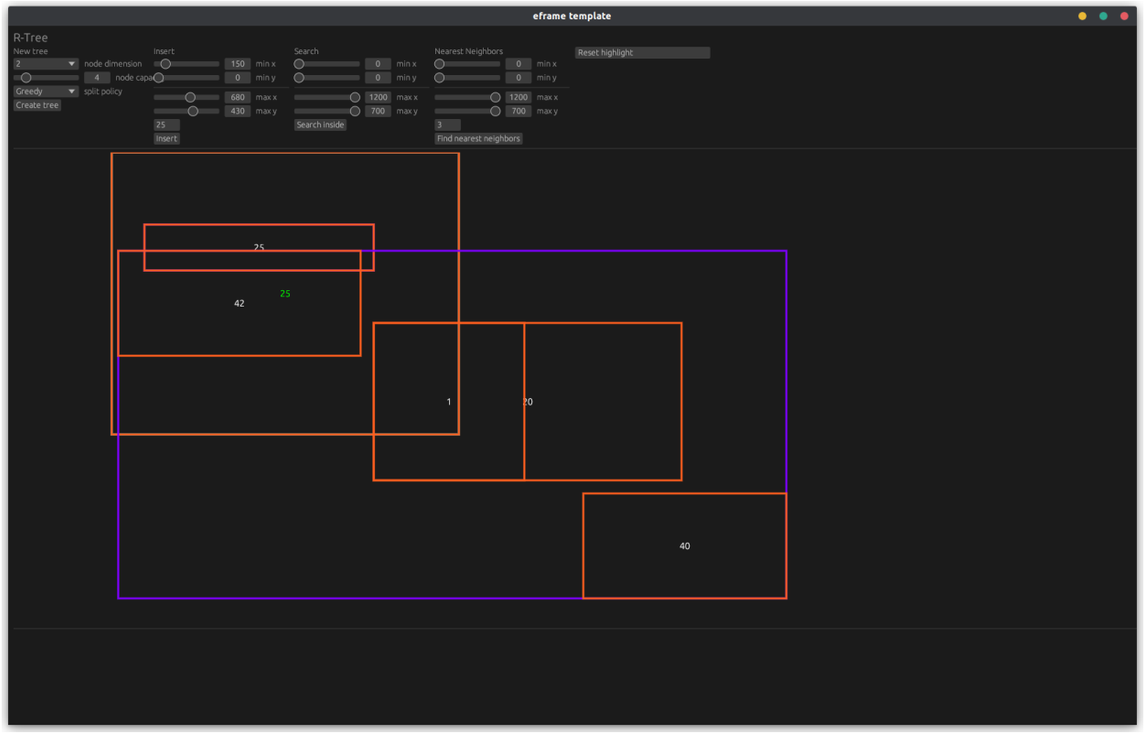
Pokyny ke spuštění aplikace se nachází v projektu v README.md.

4 Příklady vstupu a výstupu

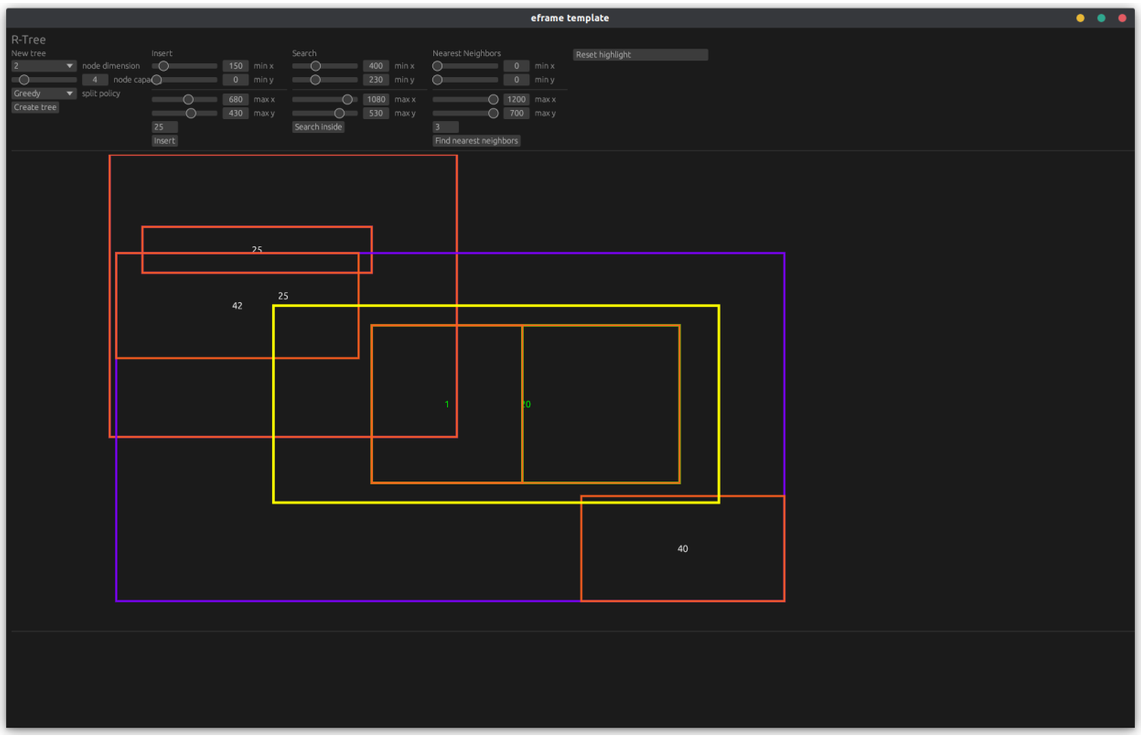
Po spuštění aplikace [1] se vyskytneme v prostředí, ve kterém můžeme nastavit a začít používat datovou strukturu. V levém horním rohu si můžeme vybrat parametry stromu a strom vytvořit. Dále je poskytnuto rozhraní ke vkládání, vyhledávání a nalezení k nejbližších sousedů. Po vložení indexů jsou zobrazeny jak listové uzly, které obsahují data, tak i vnitřní uzly stromu [2]. Dále můžeme vyzkoušet funkcionalitu vyhledání [3] a K nejbližších sousedů [4]. Nakonec můžeme aplikaci ukončit, instance stromu se automaticky uloží a při dalším spuštění načte.



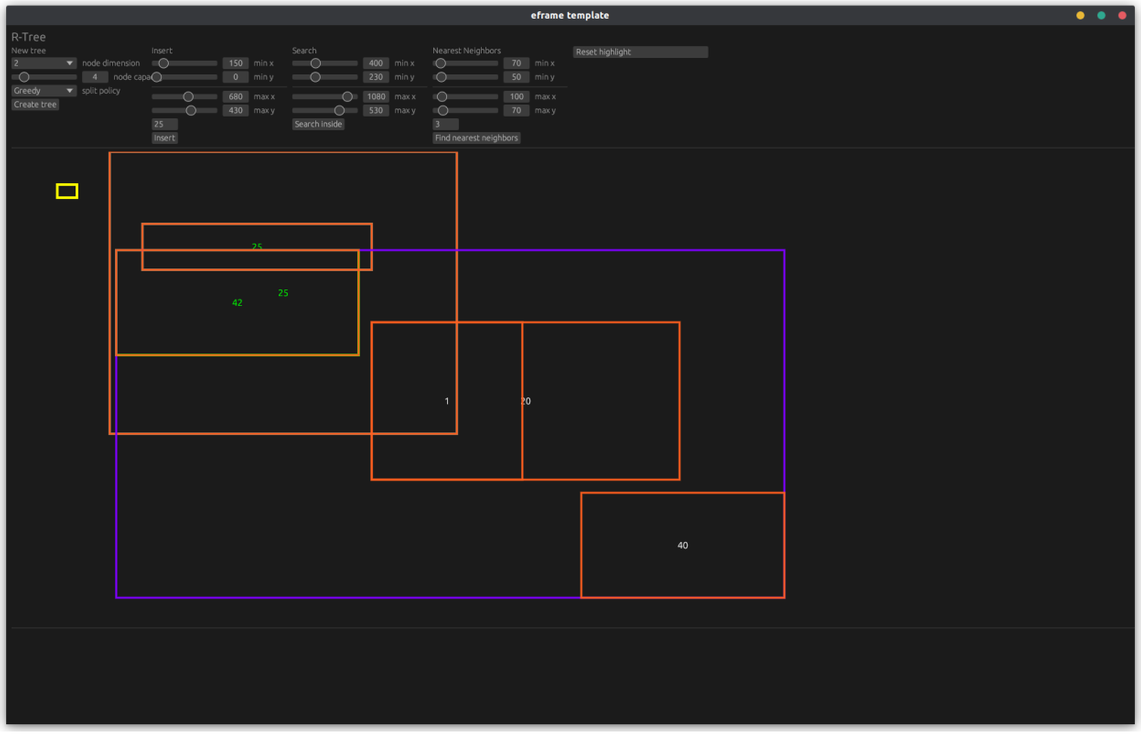
Obrázek 1: Spuštěná aplikace



Obrázek 2: Vkládání



Obrázek 3: Vyhledávání, okno rozsahového dotazu vyznačeno žlutě, nalezená data zeleně



Obrázek 4: KNN, 3 nejbližší sousedé

5 Experimentální sekce

V rámci experimentální sekce je testována především rychlost R-stromu vůči sekvenčnímu průchodu. Dále jsou také testovány různé konfigurace stromu.

Ve výchozím nastavení stromu, tedy s hladovou rozdělovací strategii uzlů, kapacitou optimalizovanou pro stránku disku a 2 dimenzionálním indexem, je strom téměř stejně rychlý, jako sekvenční vyhledávání. Vkládáním uzlů stráví o něco déle, vyhledávání je pak zhruba stejně rychlé (při konfiguraci 10,000 indexů, celkový rozsah indexů 0-10000 a polovičním rozsahem vyhledávání).

U stromu však s rychlostí velmi pomáhá, pokud jsou z širokého rozsahu vyhledávány pouze mnohem menší rozsahy. Například pro konfiguraci, kde rozsah vkládaných indexů je velmi vysoký (0-100,000), zatímco vyhledávaný je řádově nižší (0-100), je strom zhruba 10x rychlejší (0.5s pro 100,000 vyhledání z 1,000 indexů oproti 5s sekvenčně).

To samé platí pro dotaz na k nejbližších sousedů. Pokud je položen dotaz na menší množství nejbližších sousedů, je zodpovězen řádově rychleji, než v sekvenčním průchodu. Avšak se zvětšujícím se k rychle roste časová složitost a pro velmi vysoká k (taková, že se blíží celé velikosti databáze), již sekvenční průchod strom překoná. To především z důvodu, že strom při prohledávání stráví nějaký čas režii navíc, kterou sekvenční průchod téměř nemá.

Další experimenty lze prohlédnout v experimentální sekci projektu.

6 Diskuze

Implementace R-stromu je oproti sekvenčnímu průchodu efektivnější pouze za určitých předpokladů. Z experimentů jsme usoudili například to, že v případě, že vyhledáváme indexy ve vysokém rozsahu, není implementace o nic efektivnější, dokonce může být pomalejší. Pokud vyhledáváme ve velmi omezeném rozsahu, může být i řádově rychlejší.

Vkládání do stromu je pomalejší, než do sekvenční databáze. To především kvůli rozdělovacím strategiím, které mohou být poměrně pomalé. Toto je reflektováno i v implementaci, kde výchozí rozdělovací strategie je buďto hladová, která rozdělí uzel v lineárním čase s kapacitou uzlu, nebo exponenciální strategie, která vyzkouší všechny možnosti, jak uzel rozdělit. Ta se vybere pouze tehdy, je-li kapacita uzlu malá – pak je totiž zkoušení všech možností přípustné. V opačném případě, kdy je kapacita velká, není exponenciální algoritmus vhodný.

Zároveň by mohl být dobrý nápad přidat do implementace i nějakou formu globálního přeindexování – například by mohl být celý strom přepočítán v čase, kdy není zrovna využíván. Tím by pak mohla být velmi urychlena složitost vyhledávání, která na struktuře stromu velmi závisí. Při rozdělování je totiž strom vždy přeindexován pouze lokálně ve vrcholech, které jsou zrovna přeplněny a strom se tak může strukturovat do podoby, která není nejlepší možná.

7 Závěr

V této úloze byla implementována datová struktura R-strom v jazyce Rust. Bylo implementováno vkládání, vyhledávání a vyhledávání k nejbližších sousedů, dále také ukládání a načítání z disku. Dále bylo vytvořeno jednoduché grafické rozhraní, které uživateli umožňuje vyzkoušet si chování této struktury a sledovat její vnitřní stav. A nakonec byla implementace experimentálně otestována na efektivitu, kde byla porovnána se sekvenčním databází. Výstupem bylo zjištění, že v některých případech je sice struktura pomalejší, například pokud vyhledáváme velký rozsah vložených prvků, nicméně v opačném případě, kdy například vyhledáváme jeden konkrétní prvek, je struktura řádově rychlejší, než sekvenční průchod celou databází. Dále by bylo možné implementaci rozšířit například o lepší mezipaměť, další metody rozdělování přeplněných uzlů a vylepšení grafického rozhraní.

Odkazy

[1] *egui*. URL: `https://docs.rs/egui/latest/egui/`.

[2] Ph.D Ing. Jiří Novák. "BI-VWM.21 - Vyhledávání na webu a v multimediálních databázích Zadání zápočtového projektu: R-strom". In: (2023).

[3] Cheung K. L. a Fu A. W.-C. "Enhanced nearest neighbour search on the R-tree". In: (1998).

[4] *Rust*. URL: `https://www.rust-lang.org/`.